

Pontifícia Universidade Católica do Rio Grande do Sul (PUCRS) - Escola Politécnica
Inteligência Artificial - Turma 32 - Professor Silvia Maria W Moraes
Artur Cardoso, Artur Marcon, João Gabriel Viana, Luca Martin Manfroï,
Maria Eduarda Contu e Robson Lopes

Tic Tac Toe com IA

Porto Alegre, 25 de Abril de 2025

1. Introdução

O objetivo deste trabalho é desenvolver uma solução de Inteligência Artificial capaz de classificar o estado atual de um tabuleiro do jogo da velha (Tic Tac Toe) em uma das seguintes categorias: "Tem jogo", "Jogador X venceu", "Jogador O venceu" ou "Empate".

Diferente de uma IA jogadora, o foco é criar um classificador que analise a situação do tabuleiro a cada jogada. O desenvolvimento da solução envolve a preparação de dados, aplicação de algoritmos de aprendizado supervisionado, comparação de resultados e a criação de um front-end simples para interação com o sistema.

2. Análise e Preparação do Dataset

2.1 Fonte e Estrutura Original

O dataset utilizado foi obtido do repositório da UCI Machine Learning (Tic Tac Toe Endgame). Ele contém registros de partidas finalizadas de jogo da velha em um tabuleiro 3x3, onde cada amostra é composta pela descrição do estado final do tabuleiro e uma classificação associada.

As classificações possíveis no dataset original são "positive" e "negative". A classificação "positive" indica que o jogador "X" venceu a partida, enquanto a classificação "negative" representa casos em que o jogador "X" não venceu, englobando tanto vitórias do jogador "O" quanto empates. A estrutura dos dados representa o estado de cada célula do tabuleiro e o respectivo resultado final da partida.

2.2 Problemas Identificados

Durante a análise inicial do dataset, foram encontrados alguns problemas que impactam diretamente a construção da solução de Inteligência Artificial. Um dos principais foi a falta de distinção clara entre vitória do jogador "O" e empate, já que ambos os casos estavam agrupados sob o rótulo "negative".

Além disso, o dataset contemplava apenas jogos completamente finalizados, não incluindo estados intermediários em que o jogo ainda está em andamento, o que é uma necessidade importante para o projeto. Também foi observada uma distribuição desigual entre os exemplos de cada situação desejada, indicando a necessidade de balanceamento dos dados para garantir uma aprendizagem adequada dos modelos.

2.3 Adequações Realizadas

Para tornar o dataset adequado aos objetivos do trabalho, realizamos uma série de ajustes. Inicialmente, refinamos a classificação dos dados, criando novas categorias para separar explicitamente "Jogador X venceu", "Jogador O venceu", "Empate" e "Tem jogo". As instâncias classificadas como "positive" no dataset original foram rotuladas como vitórias do jogador X, enquanto as instâncias "negative" foram analisadas individualmente para serem corretamente identificadas como vitórias do jogador O ou empates.

Além disso, foram criadas novas instâncias representando jogos em andamento, com espaços em branco no tabuleiro (representados pela letra "b"), pois o dataset original continha apenas jogos finalizados. Esse processo foi essencial para permitir que a IA também aprendesse a identificar estados intermediários do jogo.

Para garantir um conjunto de dados equilibrado, selecionamos aproximadamente 400 exemplos para cada uma das quatro categorias definidas. Posteriormente, os dados foram divididos em três conjuntos: treino (70%), validação (20%) e teste (10%), mantendo a proporção entre as classes em todas as divisões.

Durante o processo de análise, balanceamento e manutenção do dataset, utilizamos o ChatGPT como ferramenta de apoio. O modelo auxiliou na identificação dos problemas encontrados, na sugestão de estratégias de balanceamento e na orientação para a criação de novos exemplos.

2.4 Problemas após modificações

Durante a fase de teste dos algoritmos, percebeu-se um padrão repetitivo em que o algoritmo não identificava corretamente alguns estados do tabuleiro, não percebendo quando um dos jogadores ganhava, ou até mesmo nunca identificando empate.

2.5 Justificativas

Essas adequações foram essenciais para permitir que o modelo classificasse corretamente não apenas finais de partida, mas também jogos ainda em andamento. A separação de vitórias de O e empates garante uma classificação mais precisa, enquanto o balanceamento de classes e a divisão dos conjuntos proporcionam uma avaliação justa dos modelos de IA.

3. Soluções com Inteligência Artificial

Durante o experimento, implementamos cinco algoritmos distintos: k-NN, MLP, Árvore de Decisão, Random Forest e XGBoost. Em todos eles, adotamos práticas comuns para garantir equidade nas comparações e padronização no fluxo de trabalho.

Todos os dados do tabuleiro foram convertidos para valores numéricos, sendo 'x' = 1, 'o' = -1 e 'b' = 0. As classes foram representadas por strings ou inteiros, dependendo da exigência da biblioteca utilizada. A divisão entre conjuntos de treino, validação e teste foi mantida fixa com a proporção 70/20/10, respeitando a distribuição balanceada entre as quatro categorias.

As métricas utilizadas na avaliação dos modelos foram: acurácia, precisão, recall e F1-score. Em todos os casos, foi utilizada a média macro para garantir uma avaliação equilibrada entre as classes. Essa uniformidade permitiu avaliar os pontos fortes e limitações de cada algoritmo com base em critérios justos.

Além disso, todos os modelos foram integrados a uma API construída com Flask. Após o treinamento, cada modelo foi salvo e vinculado a uma rota específica na aplicação. A API foi então conectada a um front-end desenvolvido com React (utilizando Vite), permitindo simular partidas de jogo da velha onde, a cada jogada, o modelo de IA classificava o estado atual do tabuleiro. Isso proporcionou uma validação prática da eficácia de cada abordagem.

Durante todo o processo, contamos com o apoio de ferramentas como o ChatGPT, os slides da disciplina e recursos de pesquisa na web. Esses materiais auxiliaram na compreensão teórica dos algoritmos, na estruturação do código e na solução de problemas práticos, respeitando sempre a regra de não gerar diretamente código com ferramentas externas.

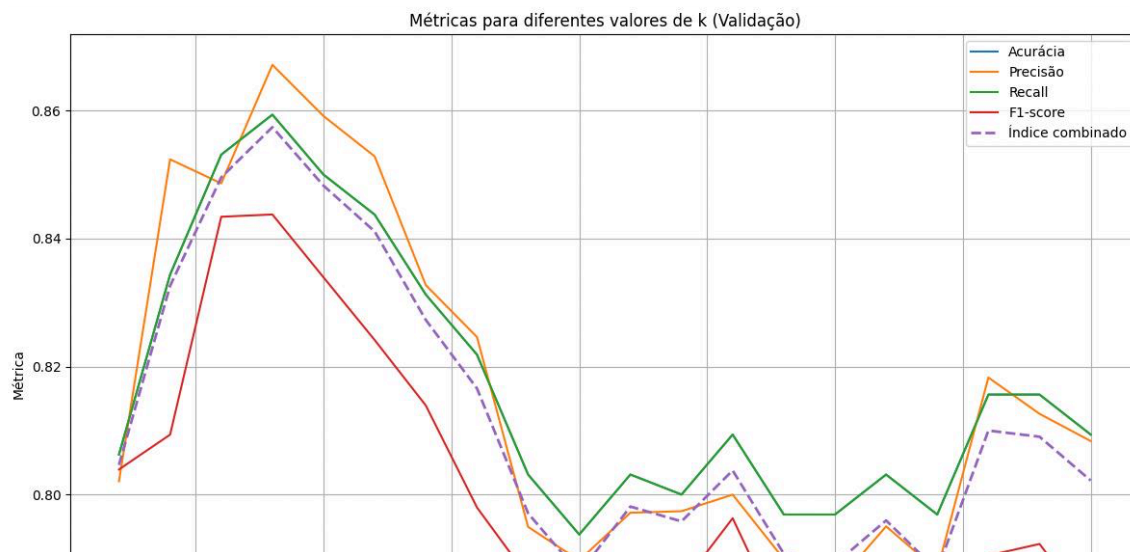
3.1 Algoritmo k-NN (K-Nearest Neighbors)

Para a construção inicial do classificador, optamos por utilizar o algoritmo k-NN, que é uma técnica de aprendizado supervisionado baseada em similaridade. Ele classifica uma nova entrada com base na maioria das classes dos seus k vizinhos mais próximos no conjunto de treinamento.

3.1.1 Etapas realizadas

- Inicialmente, os dados foram transformados de forma a permitir a aplicação do algoritmo: os valores 'x', 'o' e 'b' foram codificados como 1, -1 e 0, respectivamente.
- A variável de saída (classe) foi codificada numericamente usando LabelEncoder, permitindo que os algoritmos trabalhassem com os rótulos de forma adequada.
- Realizou-se a divisão do dataset em conjuntos separados de treino (70%), validação (20%) e teste (10%), de modo a evitar sobreposição e garantir avaliações justas entre os algoritmos.
- No conjunto de validação, testamos vários valores de k (1 a 20). Para cada k , foram calculadas as métricas de acurácia, precisão, revocação (recall) e F1-score, todas com average='macro' para garantir a avaliação balanceada entre as classes.

Esses resultados foram plotados no gráfico a seguir, para comparação visual e escolha do valor ideal de k .



O melhor desempenho foi obtido com $k = 4$, segundo nosso índice composto pela média entre as quatro métricas. Este foi o valor utilizado para treinar o modelo final.

3.2 Algoritmo MLP (Multi-Layer Perceptron)

Para expandir a análise e comparar diferentes abordagens de aprendizado supervisionado, foi implementado o algoritmo MLP (Perceptron Multicamadas), um tipo de rede

neural artificial que utiliza camadas ocultas para aprender padrões complexos a partir dos dados. A MLP é capaz de capturar relações não lineares entre atributos, sendo adequada para problemas como o do jogo da velha, onde posições no tabuleiro podem interagir de forma complexa.

3.2.1 Etapas realizadas

- Assim como na abordagem com k-NN, foi necessário realizar um pré-processamento dos dados para adaptá-los ao modelo:
- Os valores categóricos do tabuleiro foram codificados numericamente: 'x' como 1, 'o' como -1 e 'b' (vazio) como 0.
- A variável de saída (classe) já estava em formato categórico e foi mantida como string, pois o MLP Classifier aceita rótulos não numéricos.
- O modelo MLP foi configurado com os seguintes hiperparâmetros:
- Topologia: $9 \times 40 \times 4$
- Solver: 'adam', um otimizador eficiente baseado em gradiente estocástico adaptativo;
- Taxa de aprendizado inicial: 0.05;
- Momentum: 0.5, para suavizar oscilações durante o treinamento;
- Verbose: ativado para acompanhar a evolução das épocas durante o treinamento.

3.2.2 Ajuste de Hiperparâmetros

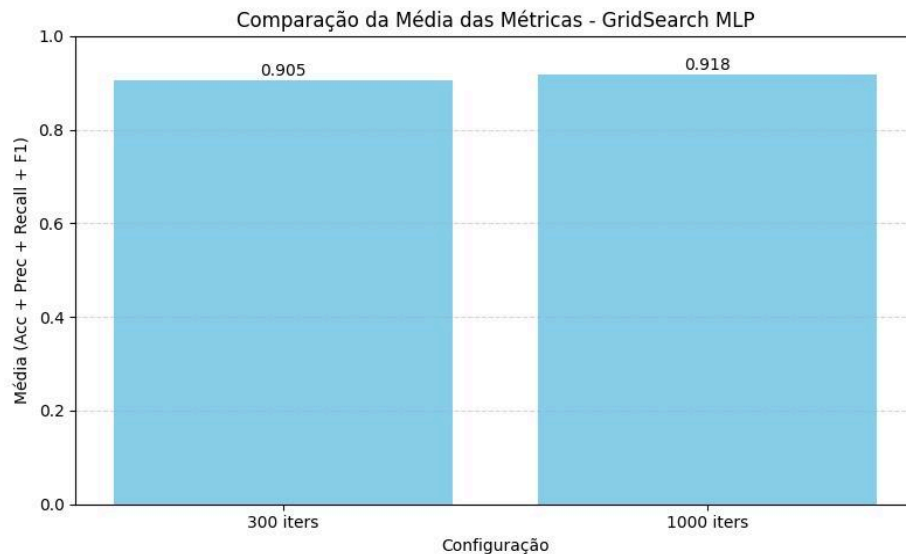
Para encontrar uma configuração adequada, utilizamos o GridSearchCV do sklearn com validação cruzada (3-fold) para testar diferentes combinações dos hiperparâmetros:

- `hidden_layer_sizes = (40,)`
- `learning_rate_init = [0.01, 0.05]`
- `momentum = [0.5, 0.9]`

Inicialmente, os testes foram realizados com o limite padrão de **300 iterações** (`max_iter=300`). Em seguida, repetimos os mesmos testes com **1000 iterações**, para verificar se um número maior de épocas traria ganho expressivo no desempenho.

Para comparar objetivamente as duas execuções, calculamos a média das quatro principais métricas: **acurácia**, **precisão**, **recall** e **f1-score**, todas com `average='macro'` para avaliação balanceada entre as classes.

O gráfico a seguir resume os resultados:



Como mostrado, a média das métricas com 300 iterações foi de aproximadamente **0.905**, enquanto com 1000 iterações alcançou **0.918**. Embora o valor absoluto tenha melhorado ligeiramente, o aumento não foi suficientemente significativo para justificar o uso de mais do que o triplo de iterações, principalmente considerando o tempo de processamento adicional envolvido.

Com base nisso, optamos por manter a configuração com 300 iterações para nossos testes finais, por ser mais eficiente e ainda assim apresentar desempenho competitivo.

3.3 Algoritmo Árvore de Decisão (Decision Tree)

Para a construção de um segundo classificador, optamos por utilizar o algoritmo de Árvore de Decisão, uma técnica de aprendizado supervisionado baseada em regras hierárquicas. Esse algoritmo realiza divisões sucessivas nos dados de entrada com base nos atributos que melhor separam as classes, formando uma estrutura em forma de árvore. A classificação de uma nova entrada ocorre ao percorrer a árvore do nó raiz até uma folha, de acordo com os valores dos atributos.

3.3.1 Etapas realizadas

- O classificador foi treinado com o critério de divisão entropy, que maximiza a informação ganha a cada divisão da árvore, buscando uma árvore mais informativa. Foi usado um valor fixo de `random_state=40` para garantir reprodutibilidade dos resultados.
- Após o treinamento, o modelo foi avaliado sobre os conjuntos de teste e validação, gerando métricas de desempenho.

3.4 Algoritmo Random Forest

Para complementar os experimentos com k-NN, MLP e Árvore de Decisão, utilizamos como quarto algoritmo o Random Forest, uma técnica poderosa e amplamente utilizada em tarefas de classificação. A Random Forest é um algoritmo do tipo ensemble, baseado na construção de múltiplas árvores de decisão, combinando suas previsões para produzir um resultado mais robusto e preciso.

3.4.1 Como funciona o Random Forest

O algoritmo Random Forest funciona criando várias árvores de decisão independentes durante o treinamento. Cada árvore é treinada com uma amostra aleatória dos dados (com substituição, processo chamado de bootstrap) e, a cada divisão de uma árvore, um subconjunto aleatório dos atributos é considerado. Essa aleatoriedade ajuda a criar árvores diversas e independentes, o que reduz a variância do modelo final e melhora sua capacidade de generalização.

Na fase de predição, cada árvore emite seu voto (classificação) e o Random Forest escolhe a classe que recebeu mais votos, ou seja, faz uma votação por maioria. Isso torna o modelo mais resistente a overfitting em relação a uma única árvore de decisão.

3.4.2 Etapas realizadas

- O pré-processamento foi o mesmo adotado nos outros algoritmos: os valores do tabuleiro ('x', 'o', 'b') foram convertidos para 1, -1 e 0, respectivamente. O conjunto de dados foi dividido previamente em treino, validação e teste (70%, 20% e 10%), assegurando que todos os modelos sejam avaliados sob as mesmas condições.
- Foi utilizado o RandomForestClassifier da biblioteca sklearn.ensemble, com os seguintes parâmetros:
- n_estimators=100: número de árvores na floresta.
- criterion='entropy': critério usado para medir a qualidade da divisão nas árvores.
- max_depth=None: sem limite de profundidade, permitindo que as árvores cresçam até suas divisões ótimas.
- random_state=42: fixação da aleatoriedade para reprodutibilidade.
- Após o treinamento com os dados de treino, o modelo foi avaliado tanto no conjunto de validação quanto no de teste. As métricas usadas foram: acurácia, precisão, recall e F1-score, todas com average='macro', para respeitar o balanceamento entre as classes.

3.5 Algoritmo XGBoost

Para complementar os experimentos com Random Forest, Árvore de Decisão e MLP, foi utilizado o XGBoost, uma técnica avançada e amplamente adotada em tarefas de aprendizado supervisionado. O XGBoost é uma implementação de Gradient Boosting, que combina múltiplos modelos de aprendizado, em sequência, para melhorar a precisão do modelo final.

Ele é conhecido por seu desempenho superior em competições de machine learning devido à sua capacidade de otimizar a precisão, reduzir o overfitting e ser altamente escalável.

3.5.1 Como funciona o XGBoost

O XGBoost baseia-se no conceito de Gradient Boosting, onde vários modelos simples (tipicamente árvores de decisão) são treinados de forma sequencial. Em vez de treinar as árvores independentemente, cada árvore subsequente tenta corrigir os erros cometidos pelas árvores anteriores, ajustando os pesos das observações de acordo com o erro residual. Esse processo de aprendizado é chamado de boosting.

As principais características do XGBoost incluem:

- Boosting com árvores de decisão: Cada árvore é treinada para melhorar a previsão da árvore anterior.
- Regulação (Regularization): XGBoost utiliza regulação L1 e L2 para controlar a complexidade do modelo e prevenir o overfitting.
- Aprendizado paralelo: O XGBoost permite treinamento em paralelo, o que acelera o processo de treinamento.
- Soma ponderada das previsões: Ao final, todas as árvores fazem uma previsão, e a previsão final é a soma ponderada das previsões de todas as árvores, com pesos ajustados durante o treinamento.

3.5.2 Etapas realizadas

- Assim como nos outros algoritmos, o pré-processamento foi realizado para transformar os valores do tabuleiro ('x', 'o', 'b') em valores numéricos, sendo 1, -1 e 0, respectivamente. O conjunto de dados foi dividido em três partes: treino (70%), validação (20%) e teste (10%).
- As classes foram mapeadas para: 'Empate': 0, 'Jogador O venceu': 1, 'Jogador X venceu': 2, 'Tem jogo': 3
- O modelo foi implementado usando o XGBClassifier da biblioteca xgboost com os seguintes parâmetros:
- `n_estimators=100`: número de árvores de decisão a serem construídas.
- `learning_rate=0.1`: taxa de aprendizado que controla o impacto de cada árvore no modelo final.
- `max_depth=5`: profundidade máxima das árvores, controlando a complexidade do modelo.
- `random_state=42`: fixação do estado aleatório para garantir a reprodutibilidade dos resultados.
- `objective='multi:softmax'`: especifica que é um problema de classificação multiclasse.
- `num_class=4`: número de classes, no caso, quatro (Empate, Jogador O venceu, Jogador X venceu, Tem jogo).
- Após o treinamento com os dados de treino, o modelo foi avaliado nos conjuntos de validação e teste utilizando as métricas de acurácia, precisão, recall e F1-score, com `average='macro'` para equilibrar o desempenho entre as classes.

3.6 Resultados e Comparações

A avaliação comparativa dos cinco algoritmos testados demonstrou que todos foram capazes de realizar classificações com desempenho aceitável, respeitando o equilíbrio entre as classes estabelecido no dataset. No entanto, alguns modelos se destacaram por sua capacidade de generalização, precisão e robustez.

O algoritmo k-NN apresentou bom desempenho com $k=4$, mas mostrou-se mais sensível a variações nos dados. Já o MLP obteve resultados satisfatórios com a topologia escolhida, sendo capaz de modelar relações complexas no tabuleiro, mas requerendo mais tempo de treinamento e ajustes finos de hiperparâmetros. A Árvore de Decisão ofereceu interpretação e simplicidade, mas com risco de overfitting.

O Random Forest foi mais robusto, com ótimo equilíbrio entre precisão e generalização, superando os anteriores em métricas como F1-score e acurácia. Entretanto, o XGBoost destacou-se como o melhor algoritmo testado. Sua abordagem sequencial de correção de erros, aliada à regulação L1/L2 e à capacidade de otimização, resultou em maior desempenho global nos conjuntos de validação e teste.

Com base nesses resultados, o XGBoost foi selecionado como o classificador final a ser utilizado na aplicação integrada ao front-end. Sua escolha foi motivada não apenas pelos números, mas pela consistência, confiabilidade e adaptação ao problema proposto, que exige sensibilidade a pequenos padrões em uma matriz 3x3 com diferentes estágios de jogo.

3.6.1 Principais dificuldades

A maior dificuldade do projeto foi balancear os dados de jogos. O trabalho exigia 400 amostras para cada classe, porém, ao utilizar um algoritmo de backtracking para gerar apenas jogos válidos, encontramos apenas 16 casos de empate possíveis.

Para entender melhor o comportamento dos algoritmos, testamos inicialmente com um dataset balanceado artificialmente e, em seguida, com um gerador que produz apenas jogos válidos, respeitando a lógica natural do jogo. Nesse segundo cenário, o balanceamento refletiu a distribuição real dos resultados em partidas autênticas.

Na tentativa de forçar um balanceamento artificial, enfrentamos uma limitação crítica: não existem 400 jogos distintos que terminam em empate seguindo as regras do jogo da velha. Isso nos levou à necessidade de incluir estados inválidos — isto é, configurações de tabuleiro que não poderiam ser atingidas em uma partida legítima — para atingir a quantidade exigida de exemplos por classe.

O resultado desses 2 datasets segue abaixo:

Dados balanceados (com exemplos válidos e inválidos) – Treinamento: 1140 | Validação: 330 | Teste: 160

Apesar de apresentarem bons resultados de precisão em dados semelhantes aos utilizados no treinamento, os modelos treinados com esse conjunto falharam com frequência ao serem testados em partidas reais. Isso ocorreu porque o treinamento incluiu jogos inválidos, o que comprometeu a capacidade dos algoritmos de generalizar corretamente para situações legítimas. A presença de dados não condizentes com as regras do jogo causou confusão nos modelos ao lidar com partidas reais, afetando negativamente a performance.

Dados válidos (desbalanceados) – Treinamento: 5400 | Validação: 400 | Teste: 201

Neste cenário, os dados utilizados foram exclusivamente válidos, mas desbalanceados — empates, por exemplo, eram pouco representados pois existe um número pequeno demais de casos reais. Como esperado, isso impactou negativamente a precisão na predição de empates. No entanto, observou-se uma melhora significativa no desempenho geral dos modelos, especialmente ao considerar apenas jogadas legalmente possíveis. As três principais classes do jogo passaram a ser corretamente reconhecidas na maioria dos casos. Destaca-se, nesse contexto, o desempenho do algoritmo MLP, que anteriormente apresentava resultados inconsistentes, mas que, com esse novo conjunto de dados, alcançou uma precisão próxima de 100%, falhando apenas em identificar alguns empates.

4. Desenvolvimento do Front-End

Para viabilizar a interação com o classificador de IA, foi desenvolvido um front-end em React (utilizando o Vite como empacotador). A interface gráfica simula o tabuleiro do jogo da velha, permitindo que um jogador humano interaja contra uma IA que realiza jogadas aleatórias. A cada jogada, o estado atual do tabuleiro é enviado à API Flask por meio de requisições POST.

A API, por sua vez, consulta o modelo correspondente (entre os cinco testados), retornando como resposta a categoria prevista para aquele estado: "Tem jogo", "Empate", "Jogador X venceu" ou "Jogador O venceu". Esse retorno é exibido dinamicamente na tela.

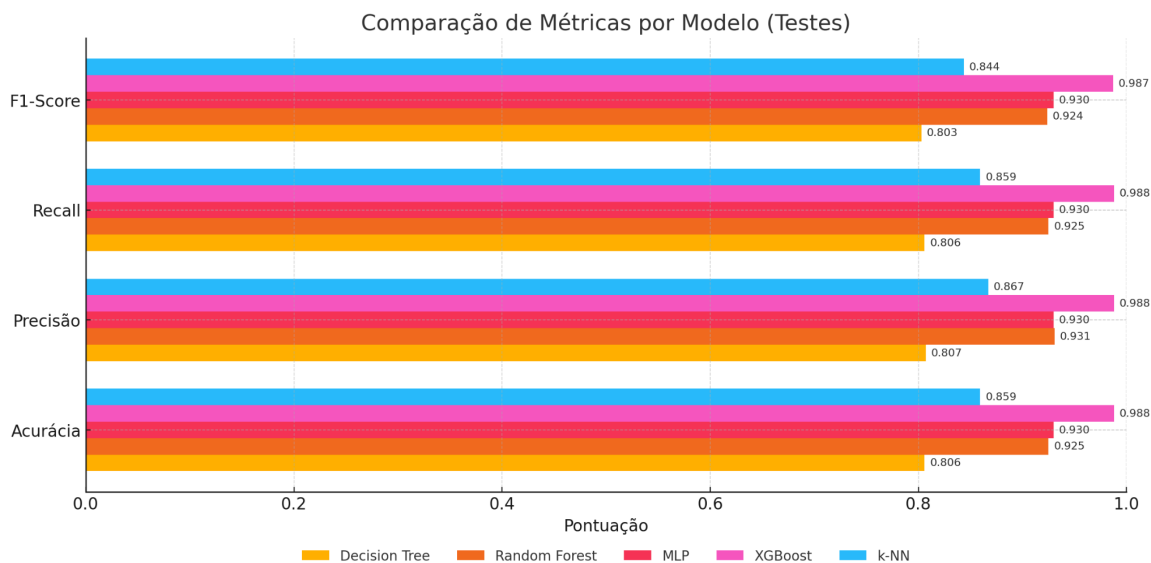
Durante o jogo, quando a IA classifica um estado como fim de partida, a interface encerra o jogo automaticamente. Caso a previsão da IA não reflita o estado real, o jogo continua e essa inconsistência é registrada. Esse processo permitiu medir a acurácia do modelo durante interações reais, fornecendo uma métrica complementar à avaliação offline.

5. Conclusão

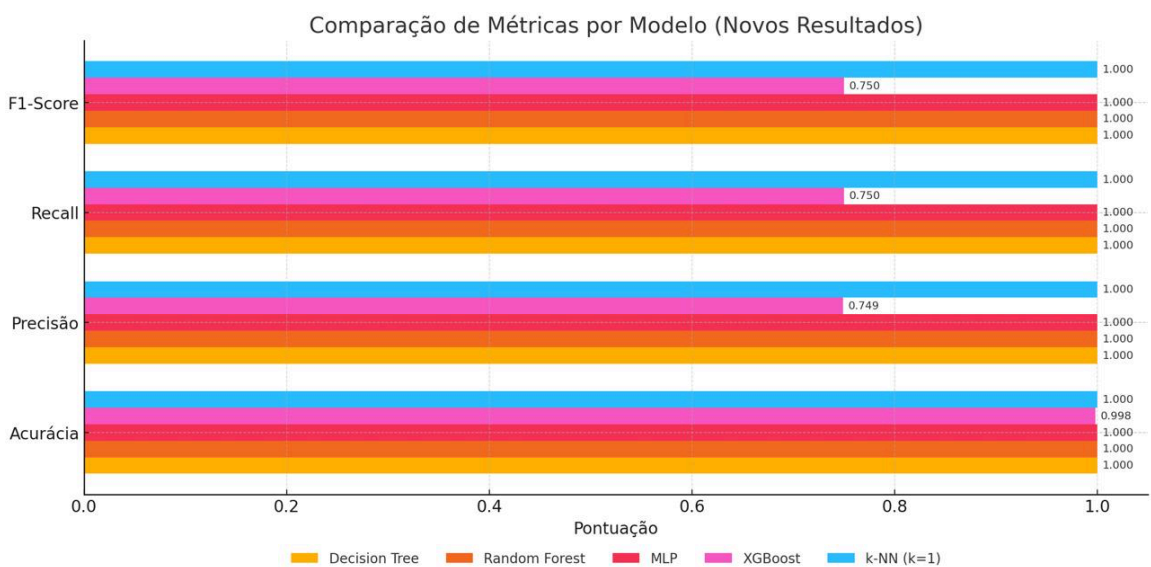
O desenvolvimento deste trabalho possibilitou a exploração prática de técnicas de aprendizado supervisionado, desde a preparação do dataset até a aplicação dos algoritmos e avaliação integrada com um sistema de front-end. A experiência permitiu entender os desafios de criar um classificador robusto para problemas com múltiplas categorias e dados discretos.

Através da aplicação de diferentes algoritmos, observou-se o comportamento individual de cada técnica, revelando suas forças e limitações. A integração com a interface gráfica trouxe uma dimensão prática valiosa, evidenciando o impacto direto de cada decisão no funcionamento do sistema como um todo.

Entre os modelos avaliados, o XGBoost demonstrou maior desempenho geral, como é possível visualizar no gráfico abaixo, comparando todas as métricas de cada algoritmo, e foi adotado como solução final, por sua robustez, precisão e adequação ao problema. O trabalho reforçou a importância de uma preparação criteriosa dos dados, testes comparativos bem estruturados e a relevância de validações práticas na aplicação de soluções de IA.



1. Comparação com 400 dados de cada classe



2. Comparação com 5400 dados